

# Lecture 6

## Transformers & Attention

AI for Geometry — MM845

Edward Hirst

Instituto de Matemática, Estatística e Computação Científica  
Universidade Estadual de Campinas

`ehirst@unicamp.br`

Agosto 2026

# Overview

1. Beyond Grids: Relational Data
2. The Attention Mechanism
3. Transformers at Scale: LLMs
4. Attention on Geometric Data
5. Summary

**Recap (Lecture 5):** Convolutions are *the* translation-equivariant linear maps.  
→ Symmetry dictated the architecture.

**Guiding question:** *What architecture suits variable-size relational data?*  
*i.e. **sets** of vertices, **sequences** of simplices,*  
*...and how did it end up inside your coding assistant?*

# The Data That Broke Previous Architectures

## Variable-size, relational data

Some examples: Text sentences, mathematical expressions, vertex lists of polytopes, sequences of simplices in a filtration...

→ Ordered or unordered collections of *tokens*: elements encoded as vectors

Why previous tools struggle:

- **MLPs**: fixed input dimension (i.e. no efficient way to handle 7 vertices today and 12 tomorrow); fixed relational structure.
- **CNNs**: built using a locality prior, but relations here are *non-local*: the two tokens that matter may be arbitrarily far apart (a variable in a bracketed expression and its exponent).
- **Recurrent networks** (historical fix): process sequences token-by-token; sequential bottleneck kills parallelism and long-range memory.

## What we want

An architecture that:

- 1 Accepts any number of tokens.
- 2 Lets every token interact with every other in one step.
- 3 Respects the set/sequence structure of the data.

⇒ **Attention** is exactly this.

# Attention: Data-Dependent Averaging

## Scaled dot-product attention

Tokens  $x_1, \dots, x_n \in \mathbb{R}^d$ .

**Example:**  $H = \frac{1}{2}(\kappa_1 + \kappa_2) \mapsto \{H, =, \times, 0.5, +, \kappa_1, \kappa_2\}$

Three learned linear maps give: *queries*  $q_i = W_Q x_i$ , *keys*  $k_i = W_K x_i$ , *values*  $v_i = W_V x_i$ .

$$\text{Att}(x_i) = \sum_{j=1}^n a_{ij} v_j, \quad a_{ij} = \frac{\exp(q_i \cdot k_j / \sqrt{d_k})}{\sum_l \exp(q_i \cdot k_l / \sqrt{d_k})} \quad \dots \text{note } \frac{e^{\alpha_i}}{\sum_j e^{\alpha_j}} \text{ is softmax activation}$$

→ This *self-attention* replaces each token vector with another vector which includes information about the rest of the sequence (other tokens contributions are weighted by importance).

- The inner product of linear actions is a *low rank* restriction.
- Softmax converts scores to weights.
- The attention weights  $a_{ij}$  depend on the *data*.
- Each of these actions is a *head*, usually multiple heads.
- Change to  $q = W_Q y_i$  (i.e. outputs) gives *cross-attention*,  $a_{ij}$  rectangular.

$$\int_M K dA = 2\pi\chi(M) - \oint_{\partial M} \kappa_g ds$$

|                      | $\int$ | $M$ | $K$ | $dA$ |
|----------------------|--------|-----|-----|------|
| $2\pi$               | .10    | .05 | .55 | .30  |
| $\chi$               | .10    | .20 | .65 | .05  |
| $M$                  | .05    | .75 | .05 | .15  |
| $\oint_{\partial M}$ | .55    | .25 | .05 | .15  |
| $\kappa_g ds$        | .05    | .10 | .25 | .60  |

Cross-attention: Each output token (row) is an average over the input tokens.

# Positional Encodings

## Permutation equivariance

For any permutation matrix  $P$ :  $\text{Att}(PX) = P \text{Att}(X)$ . Attention treats its input as a **set**.

⇒ Nothing the network computes can depend on how the tokens were indexed — order and position are simply *unavailable* to it, unless we put them into the token features ourselves.

- Perfect for unordered geometric data: vertex sets, point clouds, collections of simplices — no arbitrary ordering can be overfitted (the CNN failure mode of Lecture 5).

## Positional encodings: breaking the symmetry *by choice*

When order *does* matter: words in a sentence, time series, non-commutative terms

...give token  $t$  a position vector  $p_t \in \mathbb{R}^d$  and feed  $x_t + p_t$  into the architecture.

→ The standard choice for  $p_t$  is a bank of sinusoids at graded frequencies:

$$p_t = (\sin \omega_0 t, \cos \omega_0 t, \dots, \sin \omega_{d/2-1} t, \cos \omega_{d/2-1} t), \quad \omega_k = 10000^{-2k/d}.$$

...map sequence position to distances around the circle, information at ranges of frequencies.

→ **Learned** vectors  $p_t$  work as well!

- Same design freedom for geometry: encode coordinates, degrees, or filtration values as “positions” — symmetry is broken *by choice*, not by accident.

# The Transformer Block

## One block

Each block takes the form:

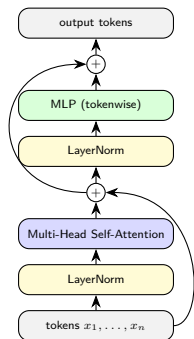
$$z = x + \text{MultiHeadAtt}(\text{LN}(x))$$

$$x' = z + \text{MLP}(\text{LN}(z))$$

...then stack blocks in sequence.

→ Each block contains many heads which each runs an attention mechanism in parallel, then concatenate — different heads learn different relations (syntax vs semantics).

- Residual connections: blocks perturb the identity; gradients flow (Lecture 5).
- LayerNorm: stabilises scale per token.
- Attention: mixes *across* tokens, so now the following MLP processes information *across* tokens.



The transformer block: residual paths (curved) carry the identity.

## Encoder blocks vs. decoder blocks

Same block; only *which*  $a_{ij}$  may be nonzero changes, masking is another deliberate symmetry break.

**Encoder:** *unmasked*, every token sees every other (Tutorial 6).

**Decoder:** *causally masked*,  $a_{ij} = 0$  for  $j > i$ , so token  $i$  sees only its past → next-token prediction (i.e. LLM).

**Encoder-decoder:** decoder adds a *cross*-attention sublayer ( $q_i$  from decoder,  $k_i \& v_i$  from encoder).

# How Transformers Become LLMs

## What makes Transformers superior?

- **Parallelism:** All tokens processed simultaneously (unlike recurrence).  
...saturates GPUs; training scales to *trillions* of tokens.
- **Long-range:** every pair interacts at every layer; no information bottleneck across distance.
- **Scaling laws:** loss falls predictably as a power law in model size, data, and compute.  
...an empirical regularity that justified the race to scale.

## LLMs in one paragraph

- The decoder block of the last slide, stacked  $\sim 100$  deep, trained on internet-scale text to *predict the next token*, via the chain rule  $p(x_1, \dots, x_n) = \prod_t p(x_t | x_{<t})$ ,  
...that narrow-looking objective is the whole distribution over text, iteration is the scaling.
  - Instruction tuning, and harnesses, then steer the continuation engine into an *assistant*.
  - Everything you experience as “understanding” in Copilot or Claude is this next-token predictor, steered.  $\implies$  intelligence is *emergent* at scale?
- **In-context learning:** New tasks can be approached without retraining: the prompt information you provide influences the attention scores at inference.
  - Connection to Lecture 1: your coding *agents* are **LLMs in a loop!**  
— generate, execute, assess, retry.

# LLMs as Tools in Mathematics

## Modern LLM uses in mathematical research

- **Code generation:** mathematics  $\rightarrow$  python/sage (your regular workflow now).
- **Literature interface:** search, summarise, translate notation, identify relevant results.
- **Formalisation:** LLM-assisted proof writing in natural language, and in type-theory languages (e.g. Lean, Roq); autoformalisation of textbook mathematics.
- **Conjecture:** identify patterns over databases of examples, generating candidate formulas to verify, suggest potential counter-examples (e.g. Jacobian conjecture).

## Areas for caution...

- **Hallucination:** invented references, plausible-sounding false statements — fluency is the failure mode's camouflage (use tools to verify).
- **Rigour Drift:** small errors and extra assumptions in long formal derivations compound (again use tools to verify).
- **Sycophancy:** assert a false lemma and it will build on it; push back on a correct answer and it may retract (be very critical and argue both ways).
- **Confidentiality:** unpublished work, referee reports, and others' preprints can be used without correct disclosure. Journals rules about LLM use vary wildly and change frequently.

Protocol (Lecture 1): every LLM output is a *conjecture* — verify by execution, proof, or independent computation before use.



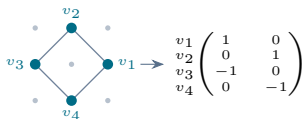
# Geometric Tokens

## Geometric Data Candidates

Anything encodable as a finite set/sequence of vectors can enter a transformer:

tokens = {polytope vertices, monomials of a defining polynomial, simplices of a complex, vertices of a graph, points of a cloud}.

Reflexive polygon  $\rightarrow$  vertex tokens



Token = lattice coordinates. Here  $d=2$ .

Plane cubic  $\rightarrow$  monomial tokens

$$f = \underbrace{y^2 z}_{m_1} - \underbrace{x^3}_{m_2} - \underbrace{x z^2}_{m_3}$$
$$\begin{matrix} m_1 \\ m_2 \\ m_3 \end{matrix} \begin{pmatrix} 0 & 2 & 1 \\ 3 & 0 & 0 \\ 1 & 0 & 2 \end{pmatrix}$$

Token = (exponents  $a_x, a_y, a_z$  | coefficient  $c$ ).

- *Genuinely unordered*: relabelling vertices, or reordering the terms of  $f$ , changes nothing — permutation equivariance is the *exact* symmetry, so **no** positional encoding.
- Both are *sets of lattice points* (polytope vertices; Newton-polytope exponents), and  $n$  varies freely — 3 and 4 here, hundreds in practice, on the same weights.
- Where order *does* matter (filtration, braid word), add positional encodings.

## Tutorial 6

A minimal set-attention model on point clouds: predict a geometric quantity (the diameter); inspect attention maps for geometric meaning.

# A Minimal Transformer in PyTorch

## Example (Set transformer)

```
import torch.nn as nn

layer = nn.TransformerEncoderLayer(
    d_model=64, nhead=4, dim_feedforward=128,
    batch_first=True,           # shapes (batch, token, feature)
    norm_first=True)           # LN before each sublayer
encoder = nn.TransformerEncoder(layer, num_layers=3,
    norm=nn.LayerNorm(64))      # final LN, pre-LN needs it
readout = nn.Linear(64, 1)     # pooled vector -> scalar

def forward(tokens):           # (batch, n_tokens, d_model)
    h = encoder(tokens)        # contextualised tokens
    h = h.mean(dim=1)           # permutation-INVARIANT pooling
    return readout(h)          # e.g. predict an invariant
```

- Mean-pooling the output tokens gives an invariant prediction from an equivariant body — the same invariant/equivariant decision as Lecture 5.
- Embed raw data to `d_model` with a small linear layer first; add positional features only if order genuinely carries information.
- Training loop: unchanged since Lecture 3.

# Summary

## Key points

- Attention is data-dependent weighted averaging built from three linear maps; all-pairs interaction at  $O(n^2)$  cost, for  $n$  the number of tokens.
- Self-attention is permutation-equivariant: transformers are architectures for *sets*, with order restored via positional encodings only when wanted.
- The transformer block = attention (mix across tokens) + MLP (process within tokens) + residuals + normalisation.
- LLMs are next-token predictors at scale; in-context learning makes them versatile, hallucination makes verification non-negotiable.
- For geometry: tokenise {vertices, monomials, simplices, etc.} into vectors; ...attention maps offer a window into what the model finds relevant.

## Next

**Tutorial 6:** Minimal attention model on point-cloud data.

**Lecture 7:** Unsupervised learning — clustering and the geometry of data without labels.